# CSE_3523

# Segment-4
# Interrupt system

# Microprocessor, microcontroller and embedded system

## Course code: CSE-3523
## Credit Hour: 3

Md. Mujibur Rahman Maruf
Lecturer, Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com



**International Islamic University Chittagong(IIUC)**

# interrupt

- **An interrupt** is either a hardware generated call (externally derived from a hardware signal) or software generated CALL (internally derived from the execution of an instruction or by some other internal event)
- Interrupt is the method of creating a temporary halt during program execution and allows peripheral devices to access the microprocessor.
- The microprocessor responds to that interrupt with an ISR (Interrupt Service Routine), which is a short program to instruct the microprocessor on how to handle the interrupt

# Types of interrupt

- **Hardware**
- an external signal is applied to the NMI input pin or the INTR input pin
- NMI - non-maskable interrupt
- INTR – interrupt used to deal with I/O devices that need attention
- **Software**
- Occurs when INT instruction is executed
- When some error condition is produced by the execution of an instruction such as divide by zero

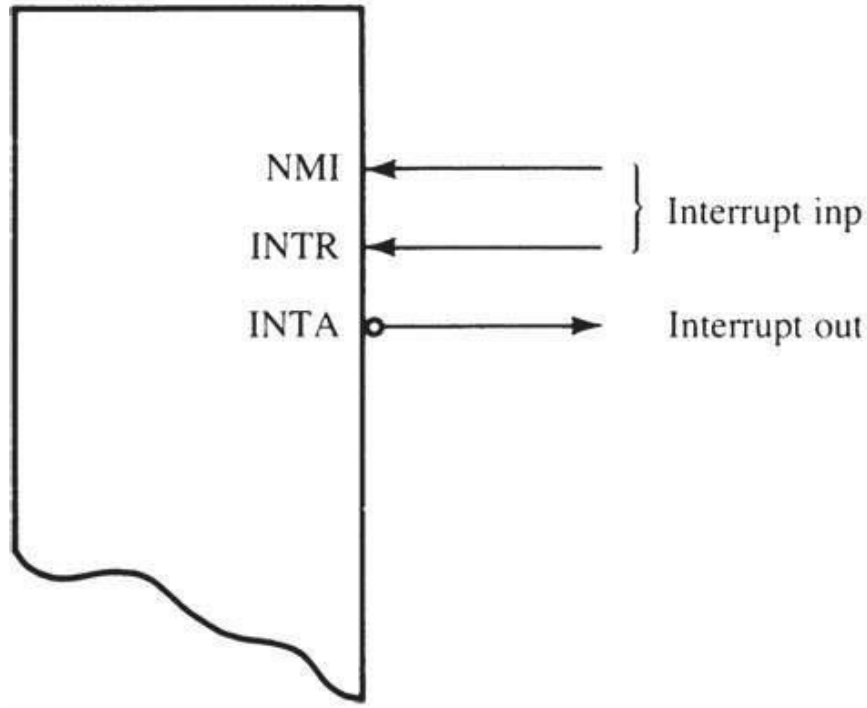
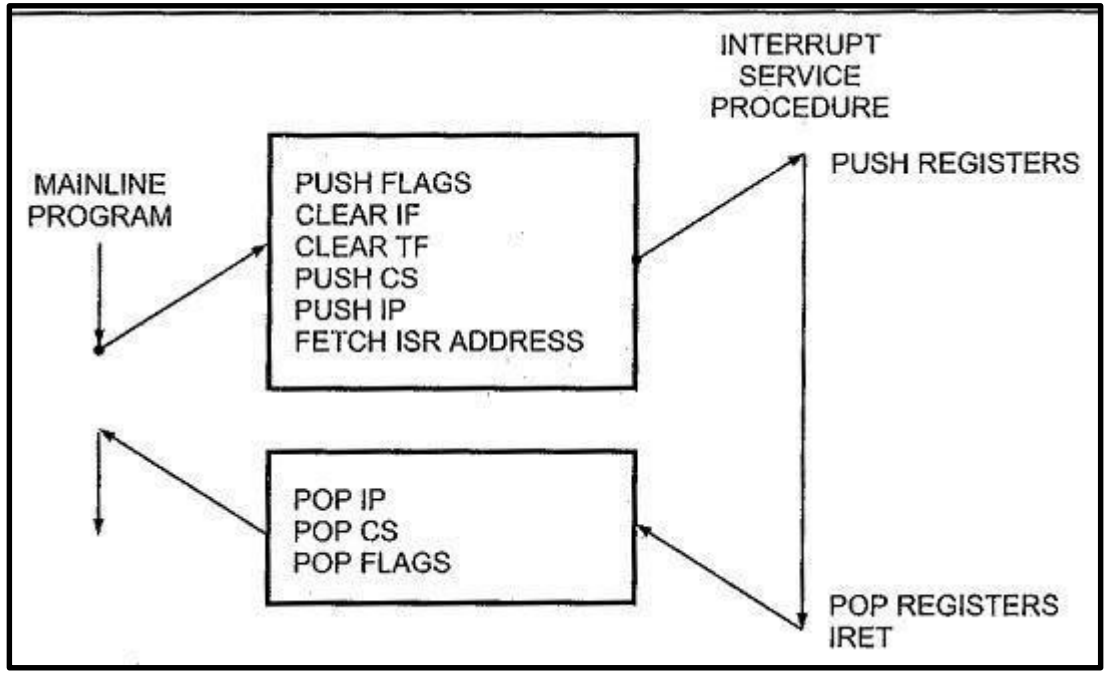



**Figure :** The interrupt pins on all versions of the Intel microprocessor.

# interrupt Handling in 8086




- When an interrupt is requested…
  - the CPU finishes executing current instruction
  - pushes flag register onto stack
  - disables the INTR input by clearing the IF (interrupt flag) in the flag register
  - clears the TF (trap flag) in the flag register
  - pushes the current CS contents onto the stack
  - pushes the current IP contents onto the stack
  - does an indirect far jump to the start of the ISR (Interrupt Service Routine)

# interrupt Handling in 8086

- Processor executes Interrupt Service Routine (ISR) like a normal procedure
- Only difference is normal procedures use RET at the end of a procedure, ISRs use an IRET instruction instead of RET
- This pops IP, CS, and the Flag registers
- ISRs can be interrupted!

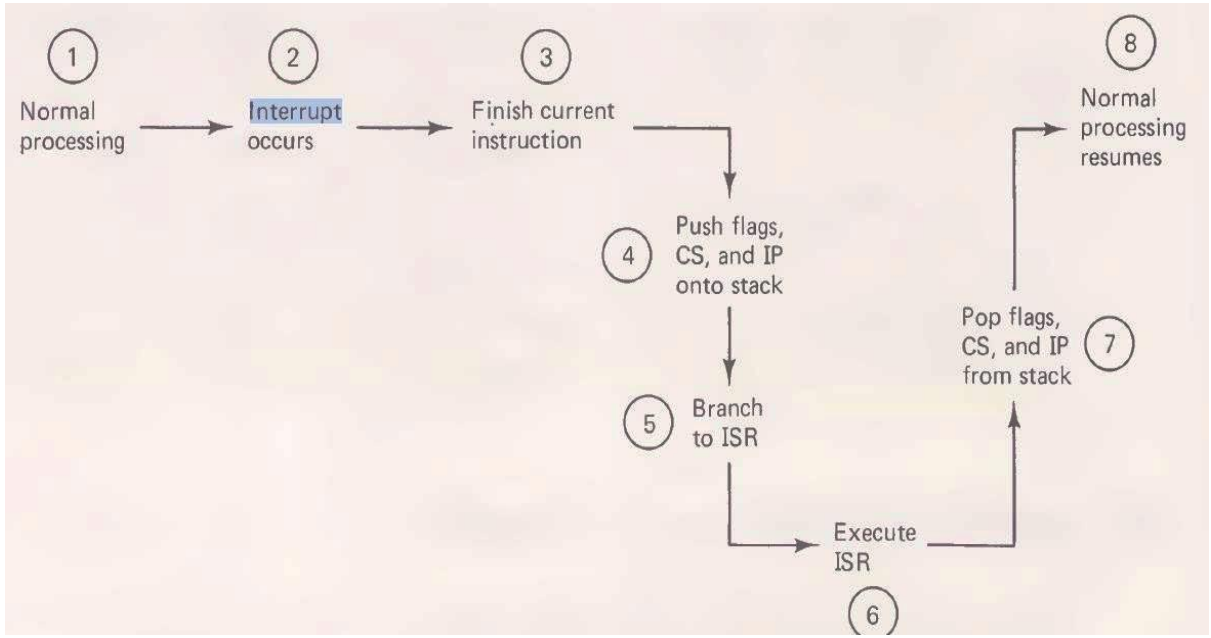
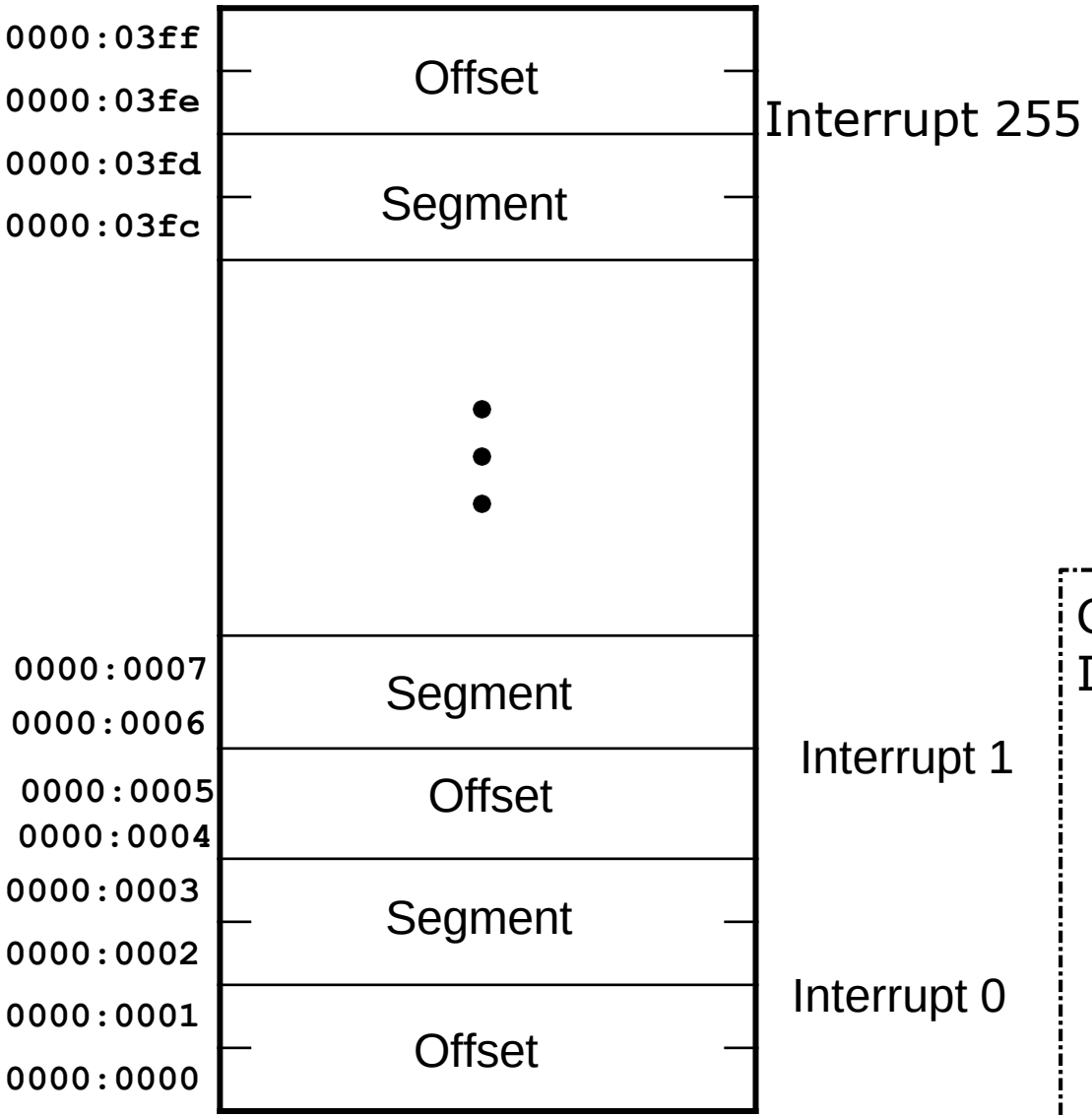



Fig:Interrupt Handling Sequence

# interrupt Vector Table

- In 8086, 1st 1k bytes of memory 00000-003ffh is set aside as a table for storing the starting addresses of interrupt service procedures(routines)
- Since 4 bytes are required to store the CS & IP values for each interrupt service procedure, the table can hold the starting addresses of up to 256 interrupts
- The starting address of an interrupt service procedure is called interrupt vector or interrupt pointer and the table is referred to as interrupt vector table or the interrupt pointer table

# IVT Format




| CS MSB |
|---|
| CS LSB |
| IP MSB |
| IP LSB |

Given a Vector, where is the ISR address stored in memory ?

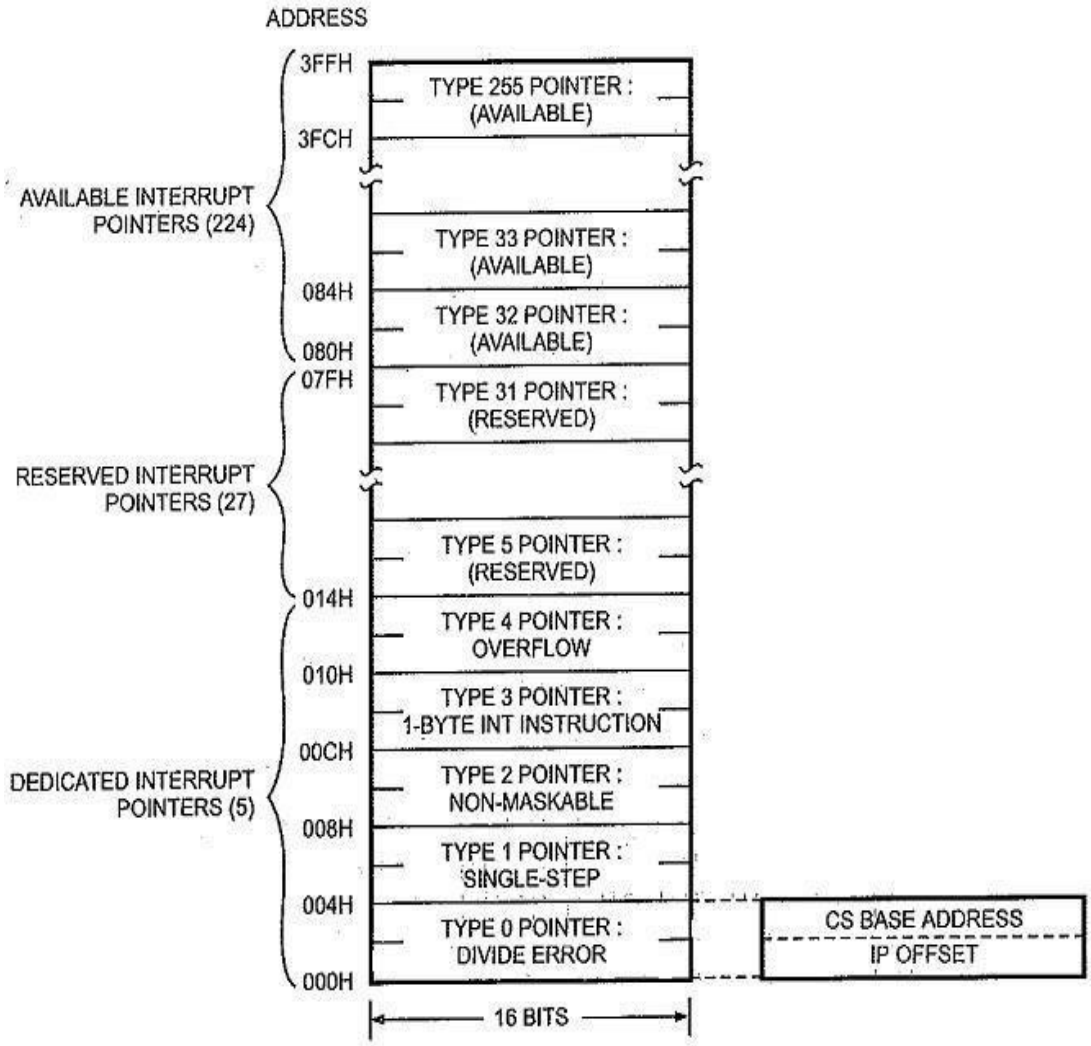
$$Offset = Type \times 4$$

Example: **`int 36h`**

Offset = (54×4) = 216
= **`00d8h`**

# interrupt Vector Table

- Each double word interrupt vector is identified by a number from 0 to 255.
- Intel calls this number, the type of interrupt
- The lowest 5 interrupts are dedicated to specific interrupts such as divide by 0, single step, NMI, interrupt caused by an instruction and overflow interrupt.



# interrupt Vector Table

- Interrupts from 5-31 are reserved by Intel for more complex processors
- The upper 224 interrupts from type 32 to type 255 are available for users to use for hardware or software interrupts
- The vector for each interrupt requires 4 memory locations.
- Therefore, when the 8086 responds to a particular type of interrupt, it automatically multiplies the type by 4 to produce the desired address in the table to get the starting address of The Interrupt Service Procedure (ISP)

# Type 0 interrupt

Occurs automatically when the result of a DIV or IDIV instruction is too large to fit in the destination register (division by zero or overflow).

```
MOV AX, 05H
MOV BL, 00H
DIV BL     ; Division by zero → Type 0 interrupt
```

**Operation:**

- The CPU pushes the Flags, CS (Code Segment), and IP (Instruction Pointer) onto the stack.
- It then loads CS and IP from memory addresses 0002h and 0000h respectively in the Interrupt Vector Table (IVT).
- The corresponding Interrupt Service Routine (ISR) handles the divide error.

Use Case:
Prevents the system from executing invalid arithmetic operations that could crash programs.

# Type 1 interrupt(Single Step )

**Cause:**

Occurs when the Trap Flag (TF) in the Flag Register is set (TF=1).
Used for single-step debugging — the CPU executes one instruction, then generates a Type 1 interrupt.

**Operation:**

- After executing each instruction, the processor automatically generates the interrupt.
- CS, IP, and Flags are pushed onto the stack.
- The ISR address is taken from 0004h–0007h in the IVT.

```
STI
MOV TF,1  ; Enable single-step
; After every instruction, CPU triggers Type 1 interrupt
```

**Use Case:**
**Used by debuggers to monitor instruction execution one at a time.**

# Type 2 interrupt:Non-Maskable Interrupt (NMI)

**Cause:**

Triggered by a low-to-high signal on the NMI hardware pin.
Cannot be disabled by software (non-maskable).

**Operation:**

The CPU completes the current instruction.

Pushes Flags, CS, and IP on the stack.

Loads new CS and IP from 0008h–000Bh in the IVT.

Executes the NMI ISR

**Example – Power Failure Protection:**

A sensor circuit detects a power drop and sends a pulse to the NMI pin.

The ISR quickly saves important data to battery-backed memory.

# Type 2 interrupt (example)

The **Non-Maskable Interrupt (NMI)** in 8086 is often used for critical system protections.

In a **boiler system**, a pressure sensor is connected to the NMI input. If the boiler pressure exceeds a preset limit, the sensor sends an interrupt signal. The corresponding Interrupt Service Routine (ISR) can immediately turn off the fuel supply, open a pressure relief valve, and sound an alarm to prevent accidents.

Another common application is power failure protection. External circuits detect the loss of AC power and send an interrupt to the NMI input. Since DC power remains available briefly (around 50 ms due to large capacitors), the ISR quickly saves critical program data to battery-backed RAM. Once AC power is restored, the system retrieves the saved data and resumes program execution seamlessly.

# Type 3 interrupt

**Cause:**
Generated by the INT 3 instruction, used by software debuggers.

**Operation:**

- When the processor executes INT 3, it automatically calls the ISR located at 000Ch–000Fh in the IVT.
- Used to pause execution at a specific point for debugging.

```
MOV AX, 5
INT 3       ; Breakpoint – stops here for debugging
MOV BX, 6
```

**Use Case:**
**Used in debugging tools to inspect register or memory contents at a specific code line.**

# Type 4 interrupt:Overflow Interrupt

**Cause:**
Triggered when the Overflow Flag (OF) is set and the INTO (Interrupt on Overflow) instruction is executed.

**Operation:**

- If OF=1, a Type 4 interrupt occurs.
- CS, IP, and Flags are pushed on the stack.
- CS and IP are loaded from 0010h–0013h in the IVT.
- If OF=0, execution continues normally (no interrupt).

```
MOV AL,108d
MOV BL,81d
ADD AL, BL    ; May set Overflow Flag
INTO          ; Type 4 interrupt if OF=1
```

Adding two signed 8-bit numbers:
+108d (01101100) + 81d (01010001) ---------- 10111101 (Overflow)
The overflow flag is set because result exceeds signed range, triggering Type 4 interrupt

**Use Case:**
**Used to detect arithmetic overflow in signed operations.**

# Type 5 interrupt: BOUND

**BOUND**

It compares a register value with a specified upper and lower bound, raising an interrupt if the value is out of bounds.

**How it Works**:

The BOUND instruction compares the contents of a register (usually AX or BX) with a specified lower bound and upper bound (stored in memory).

If the value in the register is outside the range defined by the bounds, a BOUND interrupt (Interrupt 5) is triggered.

# Type 5 interrupt: BOUND

**Cause:**
Generated when the BOUND instruction detects a register value outside specified bounds.

**Operation:**

- BOUND compares a register value with lower and upper limits in memory.
- If the value is outside the range, a Type 5 interrupt occurs.
- ISR address is stored at 0014h–0017h in IVT.

```
DATA SEGMENT
LOWER DW 10
UPPER DW 20
DATA ENDS

MOV AX, 25
BOUND AX, LOWER  ; AX=25 > 20 ⇒ Type 5 interrupt
```

**Use case:**
**Used for array index checking or ensuring variable values stay within valid limits.**

# interrupt

| Type | Interrupt Name | Trigger Source | ISR Address in IVT | Purpose / Example |
|---|---|---|---|---|
| 0 | Divide Error | DIV/IDIV overflow or divide by zero | 0000h–0003h | Division by zero check |
| 1 | Single Step | Trap Flag (TF=1) | 0004h–0007h | Step-by-step debugging |
| 2 | Non-Maskable Interrupt | NMI hardware pin (low→high) | 0008h–000Bh | Power failure, safety shutdown |
| 3 | Breakpoint | INT 3 instruction | 000Ch–000Fh | Software breakpoint for debugging |
| 4 | Overflow | INTO instruction when OF=1 | 0010h–0013h | Detect signed arithmetic overflow |
| 5 | Bound Range Exceeded | BOUND instruction (out of range) | 0014h–0017h | Array boundary checking |

# IRET – Interrupt Return Instruction

**Cause:**

Executed at the end of every Interrupt Service Routine (ISR) to return control back to the main program that was interrupted.It is not an interrupt type, but a special instruction used to end an interrupt handler.

**Operation:**

When an interrupt occurs, the processor automatically performs these actions:

- Pushes Flags, CS (Code Segment), and IP (Instruction Pointer) onto the stack.

.

# IRET – Interrupt Return Instruction

- Then it jumps to the corresponding ISR (Interrupt Service Routine).
- When the ISR finishes, it executes the IRET instruction.

**IRET** performs the reverse steps:

- Pops IP from the stack (restores the next instruction address).
- Pops CS from the stack (restores original code segment).
- Pops Flags from the stack (restores all status and control flags).

This restores the exact state of the CPU before the interrupt occurred and allows the interrupted program to continue normally

# interrupt Sources

8086 interrupt can originate from three sources:

**Hardware Interrupt**:

two hardware pins (INTR and NMI) that request interrupts…

And one hardware pin ($\overline{\text{INTA}}$) to acknowledge the interrupt requested through INTR.

**Software Interrupt**:

The processor also has software interrupts INT, INTO, INT 3, and BOUND.

Flag bits IF (interrupt flag) and TF (trap flag), are also used with the interrupt structure and special return instruction IRET

**Internal Interrupt (Exception)**:

Generated by an error during instruction execution.(divide-by-zero).

Example: If an operand is divided by zero, 8086 automatically interrupts the current program

# Hardware interrupt:NMI

- **NMI (Non-Maskable Interrupt):**
- The Non-Maskable Interrupt (NMI) has the highest priority among external interrupts in the 8086 activated on a positive edge (low-to-high transition) and is treated like a Type 2 interrupt (equivalent to executing INT 02).
- When an NMI occurs, the processor completes the current instruction before servicing the interrupt. If an internal interrupt is being serviced, it will be suspended, and the NMI will be honored, even if it is of lower priority.

# Hardware interrupt:NMI

- cannot be disabled (or "masked") by the program.
- Because NMI cannot be ignored, it’s used only for very critical situations, like:
  - Hardware memory errors.
  - Impending power failures.
- If NMIs were triggered casually, they could disrupt important operations — like disk read/write processes where interrupting midway could corrupt data.

.

# Power failure detector

**NMI as Power Failure detector**

This circuit detects a power failure and sends an interrupt (NMI - Non-Maskable Interrupt) to the processor, allowing the system to quickly save important data before complete shutdown.

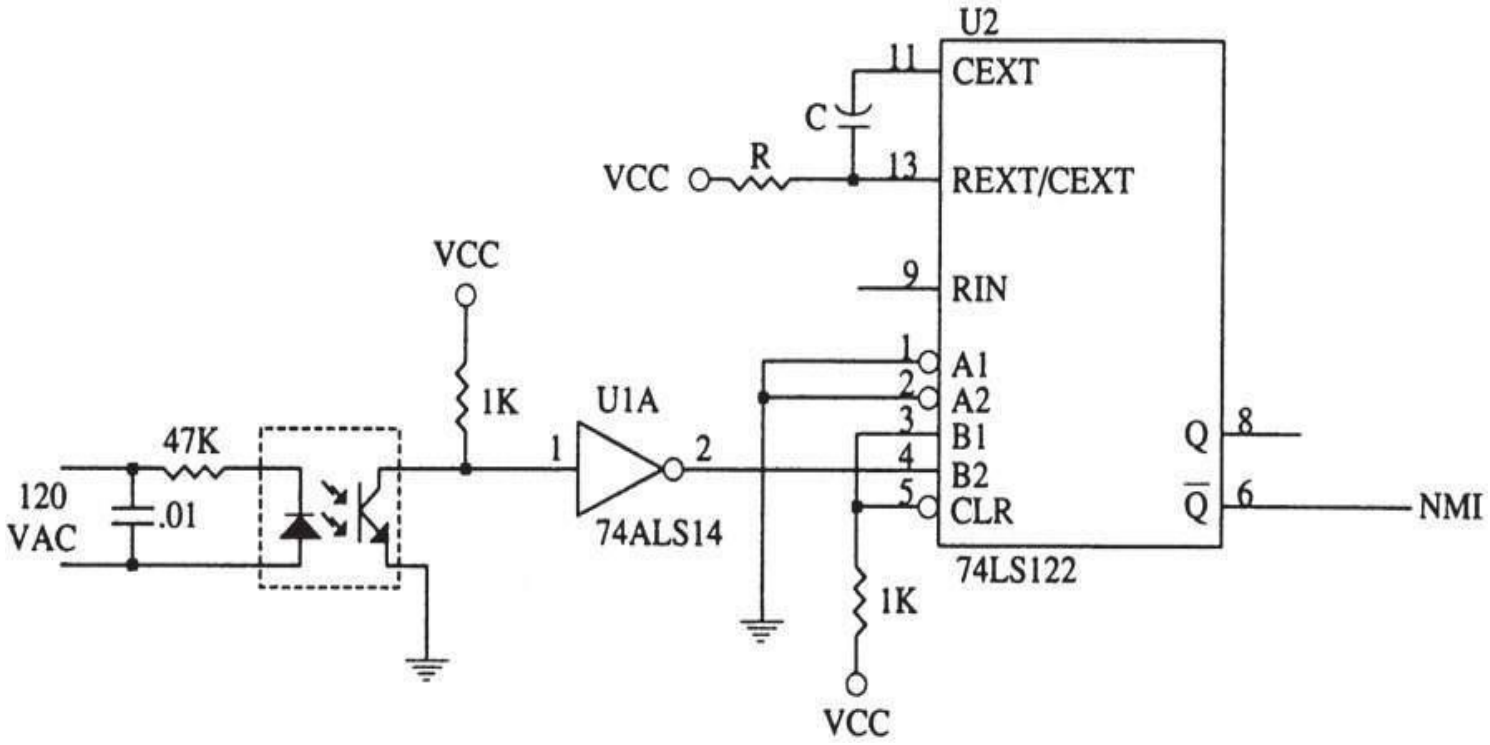



Fig:A power failure detection circuit.

# Power failure detector

**AC Power Sensing (Optical Isolator):**

120V AC powers an optical isolator; when AC is present, the internal LED turns ON, causing the output transistor to conduct.

**Inverter Stage (74ALS14 - Schmitt Trigger):**

The isolator output feeds into a 74ALS14 Schmitt Trigger inverter (U1A), which sharpens the signal. Its output goes **LOW** when AC power is present and **HIGH** when AC fails.

**Pulse Generation using 74LS122 (Monostable Multivibrator):**

The clean signal is fed into a 74LS122 one-shot (monostable) multivibrator.

When AC power fails (inverter output goes HIGH), it triggers the one-shot.

The one-shot generates a fixed-width pulse even if the AC fails completely.

# Power failure detector

**Non-Maskable Interrupt (NMI):**

Output Q’ of the 74LS122 sends a pulse to the NMI pin of the processor.

This interrupts the processor immediately (even if interrupts are disabled) to execute emergency routines (like saving important data to memory).

**Timing Components:**

The timing of the pulse width is set by R (resistor) and C (capacitor) connected to the 74LS122.

This ensures the NMI pulse is long enough for the processor to start its emergency service routine.

# Microprocessor Interrupts: INTR, INTA & Data Bus

The **INTR (Interrupt Request)** signal is sent by an external device to the microprocessor, requesting it to halt its current task and handle a more urgent operation. Upon receiving the INTR signal, the microprocessor responds by sending an INTA (Interrupt Acknowledge) signal back to the device, confirming that it has recognized the interrupt. The data bus plays a crucial role in this process by transferring the interrupt vector type number(which points to the Interrupt Service Routine or ISR) on data bus connections $D_7$–$D_0$.

# Interrupt vector type generation

**Interrupt Vector Generation Using Fixed Data**

The circuit demonstrates a simple hardware method to generate a fixed interrupt vector type (FFH) during an INTR (Interrupt Request) cycle in an 8086 microprocessor system.

- An 8-bit DIP switch (labeled 1–8) is connected to the data bus lines **D0 to D7**.

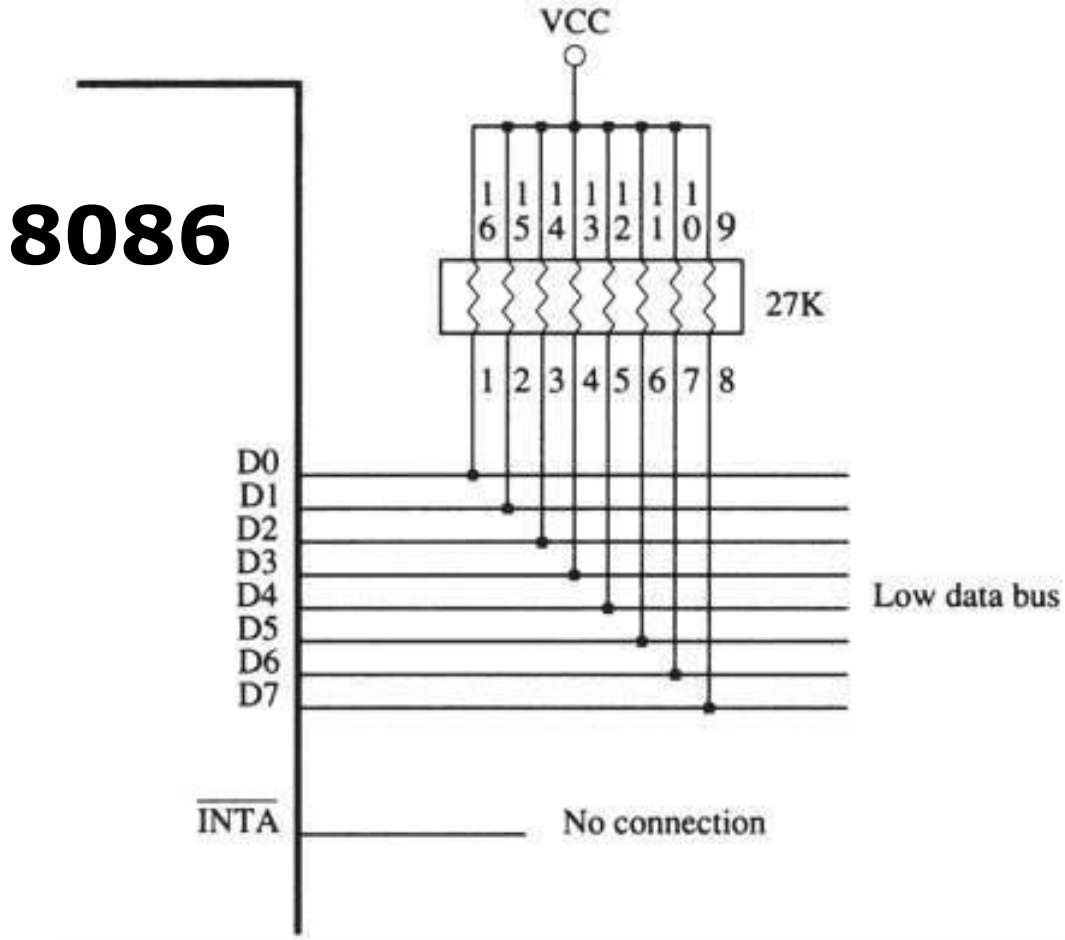



Fig: shows a circuit to appy interrupt vector type number FFH to the data bus in response to an INTR

# Interrupt vector type generation

- Each switch connects through a 27kΩ pull-down resistor to ground, ensuring that when the switch is open, the corresponding data line reads as 0.

When the switches are closed, they connect the line to VCC, forcing a logic high (1).

- In the configuration shown, all switches are closed, so all data lines are at logic high, producing the FFH value (11111111) on the data bus.
- The output is enabled in response to the INTA (Interrupt Acknowledge) signal, placing the vector on the bus only during the interrupt acknowledge cycle.
- The system does not use a buffer or decoder, so this setup always delivers the same fixed vector

# Interrupt vector type generation

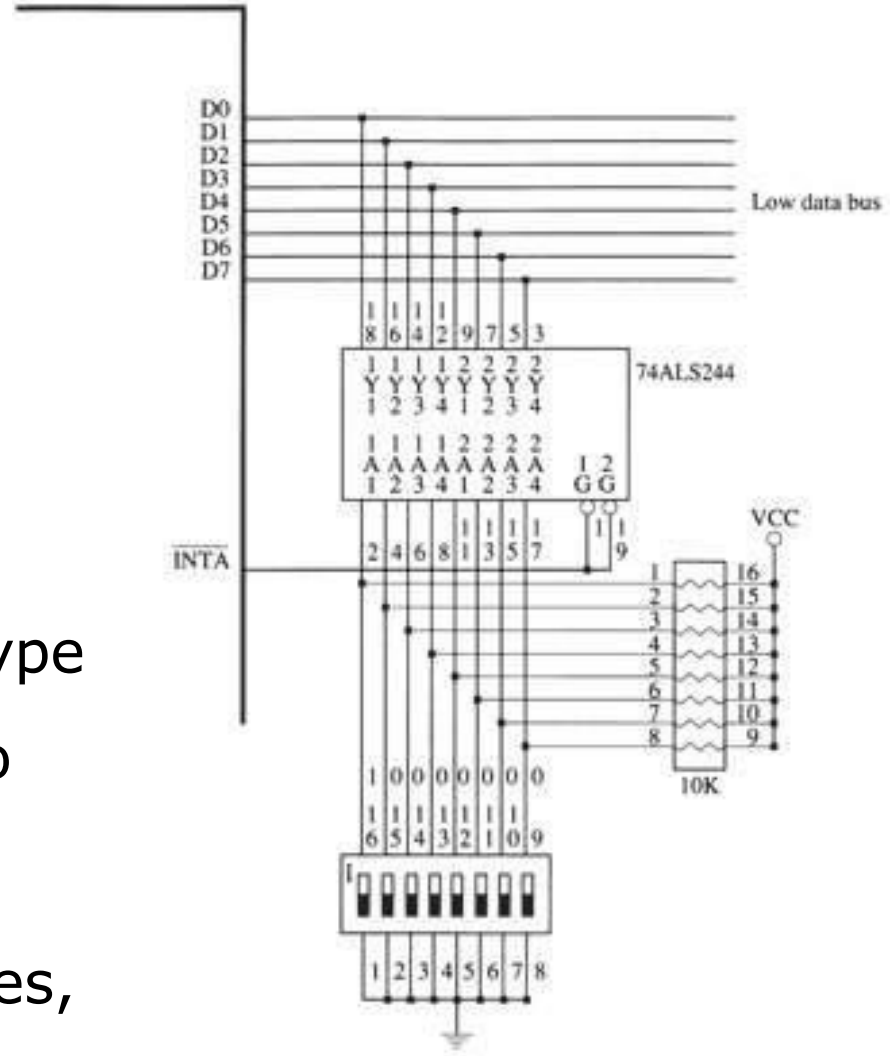
**Fixed Interrupt Vector Generator for 8086**

**(Using 74ALS244 + DIP Switches)**

This circuit provides a fixed 8-bit interrupt vector (e.g., 80H) to the 8086 during an INTR interrupt using a 74ALS244 buffer and DIP switches, controlled by the INTA signal.

**1.DIP Switches**: Set the desired 8-bit interrupt type (e.g., 1000 0000 for 80H); each switch connects to VCC or GND via 10kΩ pull-down resistors.

**2.74ALS244 Buffer**: Takes input from DIP switches, outputs to the data bus only when enabled.




A circuit that supplies a fixed interrupt vector (e.g., 80H) to the 8086 in response to INTA.

# Interrupt vector type generation

**3.INTA (Interrupt Acknowledge) Signal:**

This signal from the processor is connected to the enable pin (G1 or G2) of the 74ALS244. It enables the buffer to put the DIP switch value onto the data bus

**8086 Action**:

•. When INTR is asserted, the 8086 responds with INTA, which enables the buffer. The DIP switch value is then placed on the data bus (D0–D7) and read by the processor as the interrupt type. Each switch sets one bit via pull-down resistors. When INTA is inactive, the buffer enters high-impedance mode, leaving the bus unaffected.

•On INTR → sends INTA → reads vector from data bus → jumps to corresponding ISR.

# Expanding Interrupt

**Expanding the INTR input from one to seven interrupt request lines.**

the circuit expands one INTR input to support multiple (up to 7) interrupt request lines using a 74ALS244 octal buffer and an 8-input NAND gate. Each IR line connects to the buffer and to the NAND gate. The NAND gate monitors all IR lines simultaneously.

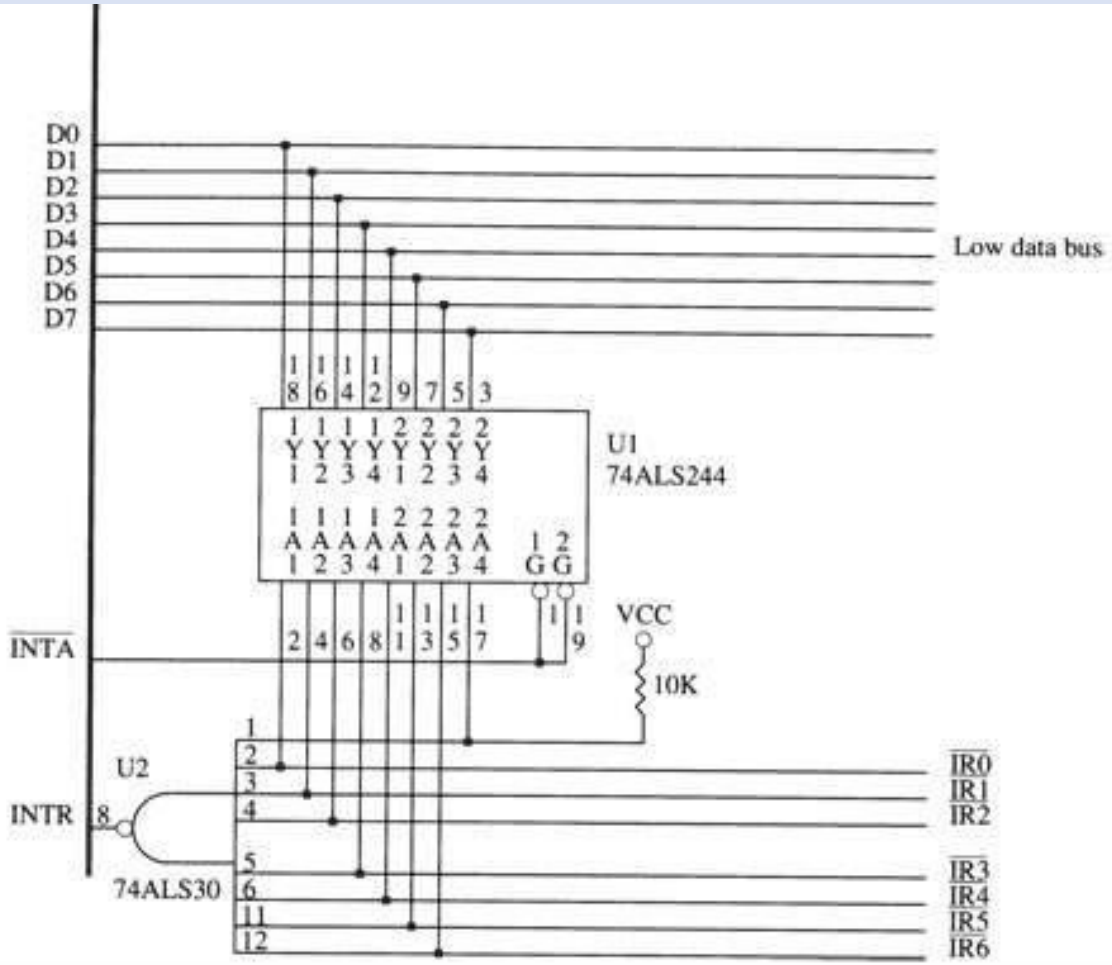



Fig: Expanding the INTR input from one to seven interrupt request lines.

# Expanding Interrupt

**Operation:**

When any IR input goes low (logic 0), the NAND gate output goes high (logic 1), triggering the INTR line of the microprocessor. The processor responds to this interrupt and reads the data from the 74ALS244 buffer to determine which IR line caused the interrupt. The appropriate interrupt vector is then fetched and executed.

# Expanding Interrupt:Daisy chain

In this setup, two 82C55 chips are used, each generating 2 interrupt signals, for a total of 4 interrupt sources. These interrupts are connected in a chain (daisy-chained), with the final signal going to the processor's single interrupt pin (INTR).

Since all interrupt outputs are combined into one line, the CPU sees only one interrupt signal and uses a single interrupt vector. To find out which device caused the interrupt, it polls each 82C55 in software. This method is hardware-efficient, using only one wire and vector, while still supporting multiple interrupt sources.

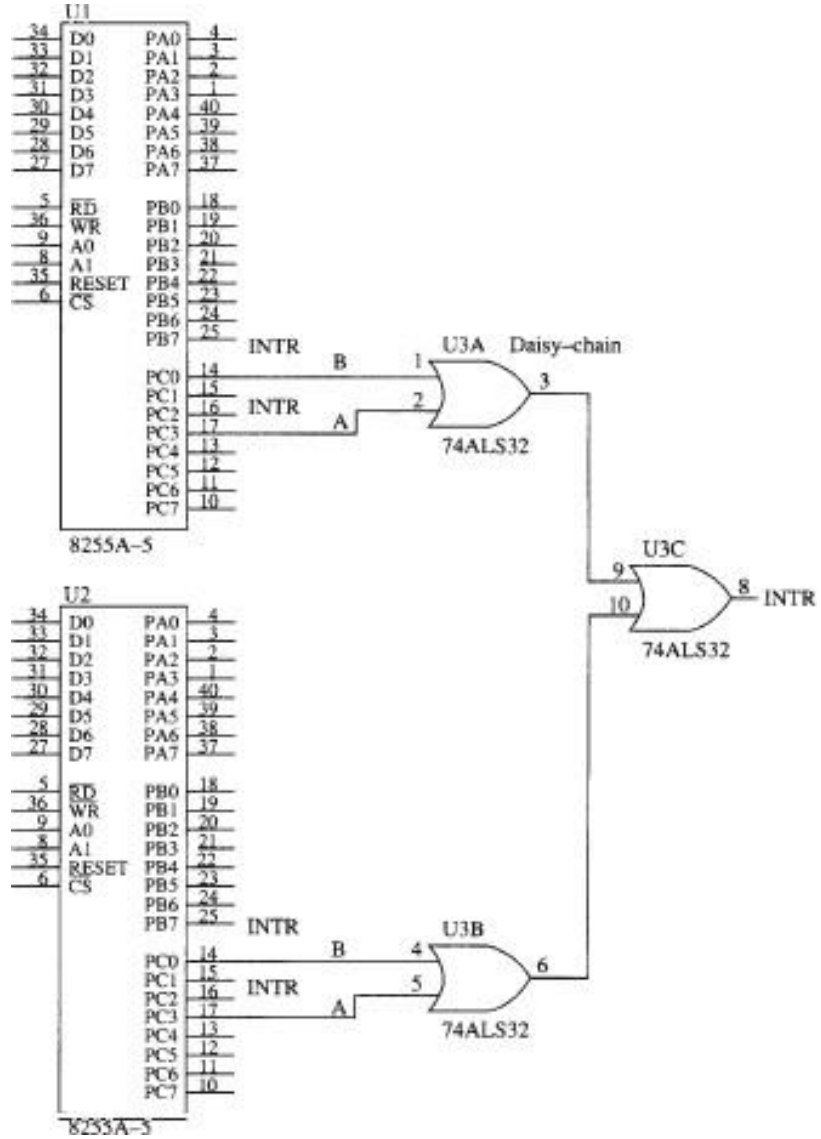



Fig:Two 82C55 PIAs connected to the INTR outputs are daisy-chained to produce an INTR signal

# Ref:

**Chapter 12**:12-1,12-2,12-3

**THE INTEL MICROPROCESSORS**

BARRY B. BREY

Eighth Edition